

Code Review #4 — recapitulare-vacanta

Code Review #4 — recapitulare-vacanta

Stadiu: 1a  · 1b  · 1c  · 1d  · 1e  (`solutie5` + `permutareMax`).

Vestea bună întâi: `solutie5` e **corectă pe orice input valid** ($n \leq 300$, numere de max 4 cifre) — am verificat-o rulând-o, nu doar citind-o. Și ideea centrală e chiar elegantă: în loc să generezi toate permutările lui `v[i]`, compari doar **permutarea maximă** — dacă cea mai mare permutare e mai mică decât toate celelalte elemente, atunci orice permutare e. Ai transformat o problemă de generare de permutări într-o problemă de sortare de cifre. Asta e gândire de algoritmist, nu de traducător de enunț.

Vestea proastă, a treia oară: review #3 spunea explicit „Nu începe 1d. Următorul commit trebuie să fie fix-ul.” Ai făcut 1d și 1e, iar cele **5 bug-uri**  **din review #1 și #2 sunt tot acolo, neatinse.** Între timp am mai găsit unul, tot în `functii.h` — deci datoria a crescut la **6**. Codul nou e din ce în ce mai bun; fundația pe care îl construiești e din ce în ce mai șubredă.

Regula pentru pasul următor

Aceeași ca în review #3, acum cu un rând în plus:

#	Bug	Unde	Din review
1	Zerorizare <code>i<9999</code> vs scanare <code>i<=9999</code>	<code>functii.h:67</code> / <code>functii.h:79</code>	#1
2	<code>valMinima</code> fără <code>return</code> pe toate căile	<code>functii.h:77-86</code>	#1
3	Numere negative sparg vectorul de frecvență	<code>functii.h:73</code>	#1
4	<code>if(n>13)</code> respinge exact cazul <code>n=13</code>	<code>solutii.h:34</code>	#2
5	<code>frecventaCifraVector</code> nu-și zerorizează <code>f</code>	<code>functii.h:88</code>	#2
6	<code>citireVector</code> e nesigură (B1 mai jos)	<code>functii.h:7-15</code>	#4 — nou

Bug-uri critice

B1 — `citireVector` nu verifică nimic (`functii.h:7-15`). Trei moduri de eșec, toate **confirmate prin rulare** (nu teorie):

1. `data.txt` cu `n > 300` → `read>>v[i]` scrie dincolo de `v[300]` — stack buffer overflow. Compilat cu AddressSanitizer, programul e omorât cu `stack-buffer-overflow`; fără sanitizer, corupe memoria în tăcere.

2. **data.txt lipsă** (șters, redenumit, sau rulezi din alt folder — clasic în Code::Blocks, unde working directory poate fi bin/) → `read>>dim` eșuează *silentios*, `dim` rămâne neinițializat. La rulare a afișat `Primul minim absolut este -2071096272`. Niciun mesaj de eroare, doar un rezultat care arată a rezultat.
3. **data.txt cu mai puține numere decât promite prima linie** (`3\n5 7`) → extragerea eșuează la EOF și `v[2]` păstrează gunoiul de pe stivă. Rezultatul devine **nedeterminist** — depinde de ce era în memorie.

Mecanismul comun: `ifstream` **nu aruncă erori** — la eșec setează un flag intern și extragerea următoare nu mai scrie nimic în variabilă. Dacă nu întrebi stream-ul cum se simte (`if(!read)`), variabilele rămân exact cum erau: neinițializate.

De ce e  și nu , deși pe `data.txt` -ul corect merge: `citireVector` e apelată din **toate cele șase** `solutieN`. Un bug într-un helper nu e un bug — e șase bug-uri. Fix-ul se face **o singură dată, în helper**, nu defensiv în fiecare apelant:

Before (`functii.h:7-15`):

```
void citireVector(int v[], int&dim)
{
    ifstream read("data.txt");
    read>>dim;
    for(int i=0; i<dim; i++)
    {
        read>>v[i];
    }
}
```

After:

```

const int NMAX = 300;

void citireVector(int v[], int& dim)
{
    dim = 0;
    ifstream read("data.txt");
    if(!read)
    {
        cout << "Nu am putut deschide data.txt" << endl;
        return;
    }
    read >> dim;
    if(dim > NMAX)
    {
        dim = NMAX;
    }
    for(int i = 0; i < dim; i++)
    {
        if(!(read >> v[i]))
        {
            dim = i;
            return;
        }
    }
}

```

Și în `solutieN`: `int v[NMAX]` în loc de `int v[300]` — cifra 300 apare acum hardcodată în șase locuri; când o schimbi într-unul singur, celelalte cinci mint.

● Issue-uri importante

1. `permutareMax` depășește `int` -ul pentru numere mari (`functii.h:260`).
`numar=numar*10+c[i]` pentru `permutareMax(1999999999)` produce signed overflow — UB;
 verificat: întoarce `1410065399` în loc de `9999999991`, iar `solutie5` afișează un răspuns greșit fără niciun semn. Pe domeniul din enunț (max 4 cifre) nu se poate întâmpla — dar **nimic nu validează** că `data.txt` respectă enunțul, iar `permutareMax` stă în `functii.h`, adică pretinde a fi helper general. Ori documentezi/validezi domeniul, ori întorci `long long`. Legat direct de B1: validarea inputului e tot o treabă a lui `citireVector`.
2. `data.txt` testează o singură ramură — pe `698 23 10 69` răspunsul se găsește la `i=2`, deci ramura `Nu exista` și bucla de back-scan (`j<i`) **nu rulează niciodată** pe niciun input din repo. Vechiul `19 39 95 19` ar fi produs exact `Nu exista` (`91≥39, 93≥19, 95≥19, 91≥19`) — l-ai înlocuit fix pe cel care testa cealaltă ramură. Și, în plus, era singurul set cu duplicate din tot repo-ul.

Regula practică: schimbă `data.txt`, rulezi pe **ambele** variante înainte de commit. Același ● 2 din review #3, nerezolvat: cod pe care nu l-ai văzut rulând e cod despre care doar *speri* că merge.

● Cleanups

1. **Două bucle copy-paste identice** (`solutii.h:92-105`). `j<i` și `j>i` au corpuri byte-identice — o singură buclă face același lucru: `for(int j=0; j<n; j++) if(j!=i && pmax>=v[j]) ok=0;`. Jumătate din cod, un singur loc de modificat. Bonus de algoritmist: „`pmax < toate celelalte`” $\equiv$ „`pmax < minimul celorlalte`” — cu cele mai mici două valori precalculate o singură dată, tot algoritmul devine $O(n)$ în loc de $O(n^2)$.
 2. **Buclele nu se opresc după verdict** (`solutii.h:96, 103`). Odată `ok=0`, nu mai poate redeveni 1 — restul comparațiilor e muncă degeaba. `break;` după `ok=0`, sau condiția `j<n && ok==1` — exact idiomul pe care îl folosești deja cu `gasit` în bucla exterioară (linia 88). Aplică-ți propriul pattern consecvent.
 3. **`c[100]` pentru cifrele unui `int`** (`functii.h:238`). Un `int` are cel mult 10 cifre; enunțul zice 4. `c[12]` spune cititorului „știi cât de mare poate fi” — `c[100]` spune „am pus ceva mare, să fie”. Mărimea unui buffer e o afirmație despre date, nu o loterie.
 4. **Indentarea minte despre structură** (`functii.h:257-261, solutii.h:106-110`). Blocul de reconstrucție a numărului e indentat ca și cum ar fi în bucla de sortare (nu e); `if(ok==1)` e indentat ca și cum ar fi în bucla `j` (nu e). Codul e corect, dar cine citește după indentare — adică toată lumea, la prima trecere — vede altă structură decât cea reală. Auto-format (Code::Blocks: Plugins → Source code formatter / AStyle) și dispare clasa asta de confuzie pentru totdeauna.
-

● Pattern-uri corecte (continuă așa)

- **Reducerea la permutarea maximă** — cea mai bună idee din tot proiectul până acum. Ai demonstrat (implicit) o echivalență matematică și ai codat-o pe cea ieftină.
 - **`permutareMax` pus în `functii.h`**, nu inline în soluție — separarea helper/soluție e acum reflex, nu regulă impusă.
 - **Idiomul `gasit==0` în condiția buclei** pentru „primul care...” — corect și curat (acum du-l și în buclele interioare, vezi ● 2).
 - **Un detaliu pe care probabil nu l-ai observat:** `permutareMax` merge corect și pe numere negative — `%` și `/` în C++ trunchiază spre zero, deci cifrele ies toate negative și sortarea descrescătoare produce exact permutarea maximă (`permutareMax(-23) = -23`, care e corect: $-23 > -32$). E corect *din întâmplare* — genul de proprietate care merită un test, ca să rămână corectă și când modifizi funcția.
-

Pașii următori

Prioritate	Ce	Notă
1	fix: bug-urile din code review 1, 2 si 4	tabelul de sus, toate 6; B1 are fix-ul scris mai sus
2	Rulează <code>solutie5</code> pe vechiul <code>data.txt</code> (4 / 19 39 95 19)	trebuie să afișeze <code>Nu exista</code> — prima testare a ramurii negative
3	1f, 1g — interschimbări min/max	vei modifica vectorul: amintește-ți că <code>solutie2</code> deja face <code>v[pozMa]=0</code> și pierde originalul

Sinteză

Criteriu	#1	#2	#3	Acum
Progres culegere	1a	1a-1b	1a-1c	1a-1e
Bug-uri critice noi în commit	3	2	0	0
Bug-uri critice deschise total	3	5	5	6
Format commit				
Testare proprie a edge case-urilor				regres

Două commit-uri la rând fără bug nou în logica proprie — algoritmica ta a ajuns înaintea igienei tale de proiect. Diferența dintre un elev bun și un programator bun nu e soluția la 1e; e ce faci cu tabelul de 6 rânduri de mai sus. Review #5 începe cu el.

Q&A — verifică-te

1. Ce afișează `solutie5` pentru `data.txt` cu 1 și 7 ? E răspunsul corect după enunț? (Indiciu: cu ce „rest al vectorului” se compară un element singur?)
2. De ce e suficient să compari doar **permutarea maximă** a lui `v[i]` cu celelalte elemente, în loc de toate permutările? Formulează argumentul într-o propoziție.
3. Rulezi executabilul dintr-un folder în care nu există `data.txt` . Programul afișează un număr. De unde vine numărul ăla și de ce nu primești nicio eroare?